

Rumie AI MVP: Frontend Integration Guide

Connecting the Mobile App to the Backend API

Prepared for the Frontend Developer

September 29, 2025

Non-Technical Overview with Technical Details for Mobile Development

Introduction

This guide helps you, the frontend developer, integrate the Rumie AI mobile app with the backend API. The backend is a secure, scalable FastAPI server handling user data, chat, integrations, and AI responses. It uses JWT authentication, Redis for session context, and Gemini API for empathetic replies.

The backend is 95% ready, with all features as API endpoints. Focus on: - Authentication (JWT for user sessions). - Calling endpoints for features (e.g., chat, MOM, integrations). - Handling errors, rate limits, and context (e.g., chat history). - Paid/free gating (check user is_paid via JWT).

The app must handle 100-200 users, so use efficient calls (e.g., caching responses locally).

Backend Overview

- **Base URL**: http://localhost:8000 (dev); Heroku URL for prod (e.g., https://rumie-mvp-backend.herokuapp.com). - **API Version**: All routes prefixed with /v1/. - **Authentication**: JWT bearer token (from /v1/auth/token). Required for all endpoints except public (e.g., /health). - **Rate Limiting**: 5-30 requests/min per endpoint (free: lower; paid: higher). - **Error Handling**: Standard HTTP codes (401 unauthorized, 429 rate limit, 500 server error). - **Context Maintenance**: Use /v1/chat/context/user_id to store/retrieve chat history (appended to Gemini prompts for empathy).

Authentication Flow

Use JWT for user sessions.

Login/Register

- **Login (POST /v1/auth/token)**:

- Response: "access_token": "jwt.token.here", "token_type": "bearer" - Store token in app storage (e.g., AsyncStorage in React Native).

- **Register (POST /v1/auth/register)**: Similar to login, auto-logs in with token.

Logout (POST /v1/auth/logout)

- Headers: Authorization: Bearer <token> - Clears token locally; blacklists on backend.

Refresh Token (POST /v1/auth/refresh)

- Headers: Authorization: Bearer <token> - Returns new token.

Best Practices

- Use interceptors (e.g., axios) to add token to headers. - Handle 401 (refresh token or re-login). - Secure storage: React Native Secure Storage.

Feature Integration

Call endpoints with JWT in headers.

1. Google Workspace & Notion

- **Get Google Data (GET /v1/integrations/google/user_id)**: Returns mock/real Gmail/Calendar data. - Example Response: "gmail": [{"subject": "Urgent"}, {"calendar": [{"title": "Meeting"}]} - **Get Notion Data (GET /v1/integrations/notion/user_id)**: Returns mock/real tasks. - **Integration**: Call on app load; display in chat (e.g., "Urgent email—reply?"). - **Rate Limit**: 5/min free.

2. MOM Agent

- **Analyze Meeting (POST /v1/mom/analyze/user_id)**: Send transcript; returns summary/highlights. - Request: "transcript": "Meeting text" - Response: "summary": "Gemini analysis", "highlights": [{"section": "Overview"}] - **Integration**: Record audio (mobile mic), transcribe (mock STT), send to endpoint; display summary with timestamps. - **Rate Limit**: 5/min free (30 min/meeting limit).

3. Empathetic Chat with Voice

- **Get Chat Response (GET /v1/chat/response/user_id?prompt=Test)**: Gemini response with mode/trait. - **Get/Set Context (GET/POST /v1/chat/context/user_id)**: Manage chat history (list of messages). - **Voice Command (POST /v1/voice/command/user_id)**: Send transcript; Gemini processes. - **Integration**: Text chat: Send prompt, append response to history. Voice (paid): Record, transcribe (mobile STT), send to /voice/command; display response. - **Rate Limit**: 20/min for chat.

4. Custom Personality Traits

- **Validate Trait** (GET /v1/users/traits/validate/trait): Check free/paid. - **Available Traits** (GET /v1/users/traits/available): List all traits. - **Integration**: On signup, validate trait; display free (2) vs paid (3).

5. Productivity Timer

- **Get Timer** (GET /v1/productivity/timer/user_id): Saved time message. - **Integration**: Track app actions (e.g., email handled), call endpoint for insights.

6. Social Media Analytics

- **YouTube Analytics** (GET /v1/analytics/youtube/user_id): Mock/real data with Gemini insights. - **Twitch Analytics** (GET /v1/analytics/twitch/user_id): Similar. - **Integration**: Connect via OAuth (mobile webview), display insights.

7. Bulk Campaign Utility

- **Run Campaign** (POST /v1/campaign/run/user_id): Send recipients/message. - **Integration**: Upload contacts, send mock/real campaigns.

General Best Practices

- **Headers**: Authorization: Bearer <token> for all. - **Error Handling**: Catch 401 (re-login), 429 (retry later), 500 (show error). - **Context**: Load /chat/context on app start; update after each response. - **Paid Gating**: Check /auth/me for is_paid; disable paid features if false. - **Mobile-Specific**: Use React Native AsyncStorage for token; Axios for requests; react-query for caching. - **Scalability**: Cache responses locally (e.g., 10 min TTL); batch calls. - **Testing**: Test with JWT; use ngrok/Heroku for mobile.

Deployment & Monitoring

- **Base URL**: Heroku app URL. - **Health Check**: GET /health for backend status. - **Monitoring**: Check backend logs; add app analytics (e.g., Firebase free tier).

This guide ensures smooth integration—contact for refinements.